

```
# =====  
# STROKE PREDICTION PROJECT - R VERSION  
# =====  
  
# 1. IMPORT LIBRARIES  
library(tidyverse)  
library(caret)  
library(randomForest)  
library(corrplot)  
library(pROC)  
library(themis) # For SMOTE  
  
# 2. LOAD DATASET  
df <- read.csv("healthcare-dataset-stroke-data.csv", stringsAsFactors = FALSE)  
  
cat("First 5 Rows:\n")  
print(head(df))  
  
cat("\nDataset Shape:\n")  
print(dim(df))  
  
# 3. DATA UNDERSTANDING  
cat("\nDataset Information:\n")  
str(df)  
  
cat("\nMissing Values:\n")  
print(colSums(is.na(df)))  
  
cat("\nStatistical Summary:\n")  
print(summary(df))
```

#### # 4. DATA CLEANING

# Fill missing BMI values with mean

```
df$bmi[is.na(df$bmi)] <- mean(df$bmi, na.rm = TRUE)
```

# Remove unnecessary column

```
df <- df %>% select(-id)
```

#### # 5. EXPLORATORY DATA ANALYSIS

# Stroke Distribution

```
ggplot(df, aes(x = as.factor(stroke))) +  
  geom_bar(fill = "steelblue") +  
  labs(title = "Stroke Distribution", x = "Stroke", y = "Count")
```

# Age vs Stroke

```
ggplot(df, aes(x = as.factor(stroke), y = age)) +  
  geom_boxplot(fill = "tomato") +  
  labs(title = "Age vs Stroke", x = "Stroke", y = "Age")
```

# Hypertension vs Stroke

```
ggplot(df, aes(x = as.factor(hypertension), fill = as.factor(stroke))) +  
  geom_bar(position = "dodge") +  
  labs(title = "Hypertension vs Stroke", x = "Hypertension", fill = "Stroke")
```

# Correlation Heatmap

```
numeric_df <- df %>% select(where(is.numeric))  
corr_matrix <- cor(numeric_df)  
corrplot(corr_matrix, method = "color", addCoef.col = "black", tl.col = "black")
```

#### # 6. ENCODE CATEGORICAL DATA

```
categorical_columns <- c('gender', 'ever_married', 'work_type', 'Residence_type', 'smoking_status')  
df[categorical_columns] <- lapply(df[categorical_columns], function(x) as.numeric(as.factor(x)))
```

```
df$stroke <- as.factor(df$stroke)
```

## # 7. TRAIN-TEST SPLIT

```
set.seed(42)
```

```
trainIndex <- createDataPartition(df$stroke, p = 0.8, list = FALSE)
```

```
train_data <- df[trainIndex, ]
```

```
test_data <- df[-trainIndex, ]
```

## # 8. HANDLE CLASS IMBALANCE & FEATURE SCALING (Using Recipes)

```
library(recipes)
```

```
recipe_obj <- recipe(stroke ~ ., data = train_data) %>%
```

```
  step_smote(stroke) %>%
```

```
  step_center(all_predictors()) %>%
```

```
  step_scale(all_predictors()) %>%
```

```
  prep()
```

```
train_processed <- bake(recipe_obj, new_data = NULL)
```

```
test_processed <- bake(recipe_obj, new_data = test_data)
```

```
cat("\nClass Distribution After SMOTE:\n")
```

```
print(table(train_processed$stroke))
```

## # 11. MODEL BUILDING

### # Logistic Regression

```
lr_model <- glm(stroke ~ ., data = train_processed, family = "binomial")
```

```
lr_prob <- predict(lr_model, test_processed, type = "response")
```

```
lr_pred <- ifelse(lr_prob > 0.5, 1, 0)
```

### # Random Forest

```
rf_model <- randomForest(stroke ~ ., data = train_processed, ntree = 100)
```

```
rf_pred <- predict(rf_model, test_processed)
```

#### # 12. MODEL EVALUATION

```
cat("\n=====\\n")
```

```
cat("LOGISTIC REGRESSION RESULTS\\n")
```

```
cat("=====\\n")
```

```
print(confusionMatrix(as.factor(lr_pred), test_processed$stroke))
```

```
cat("\n=====\\n")
```

```
cat("RANDOM FOREST RESULTS\\n")
```

```
cat("=====\\n")
```

```
rf_cm <- confusionMatrix(rf_pred, test_processed$stroke)
```

```
print(rf_cm)
```

#### # 13. CONFUSION MATRIX VISUALIZATION

```
plt_cm <- as.data.frame(rf_cm$table)
```

```
ggplot(plt_cm, aes(Prediction, Reference, fill = Freq)) +
```

```
  geom_tile() + geom_text(aes(label = Freq)) +
```

```
  scale_fill_gradient(low = "white", high = "steelblue") +
```

```
  labs(title = "Confusion Matrix (Random Forest)")
```

#### # 14. ROC-AUC CURVE

```
rf_prob <- predict(rf_model, test_processed, type = "prob")[,2]
```

```
roc_obj <- roc(test_processed$stroke, rf_prob)
```

```
cat("\nROC-AUC Score:", auc(roc_obj), "\\n")
```

```
plot(roc_obj, col = "blue", main = "ROC Curve - Random Forest")
```

```
abline(a = 0, b = 1, lty = 2, col = "red")
```